EEE4120F HPES 2026
YODA Project Report

A CORDIC Based Coprocessor for Trigonometric Acceleration on StarCore1

Joshua Smith · Ebrahim Bhyat · Tlangelani Tembe

May 2026

I. Abstract

This paper presents the design and implementation of a Coordinate Rotation Unit (CRU) as a hardware coprocessor extension to the StarCore1, a 16 bit single cycle processor. Satellite attitude determination requires the computation of trigonometric functions (sine and cosine) of Euler angles, a task that is prohibitively expensive in software on a processor lacking a multiply instruction. The proposed CRU leverages the CORDIC (Coordinate Rotation Digital Computer) algorithm to compute both sine and cosine using only additions, subtractions, and arithmetic bit shifts, operations that map efficiently onto FPGA fabric.

The coprocessor is integrated via a reserved opcode (1010, COP instruction) in the StarCore1 ISA. A quadrant folding preprocessing stage extends CORDIC's native $\pm\pi/2$ convergence domain to $\pm\pi$. The engine executes 15 CORDIC iterations with a fixed 19-cycle latency; the CPU stalls during computation by freezing the program counter.

The cosine result is written directly to a destination register, while the sine result is made accessible via a memory mapped intercept at address 7.

Simulation results confirm correctness across the full angular range, with the quantization error bounded to ≈0.006%. The hardware integration achieves an exact latency of 19 clock cycles, representing an ≈90% reduction in execution time compared to a software only Taylor series implementation.

II. Introduction

Modern satellite systems rely on continuous attitude determination to orient solar panels, antennas, and sensors. The attitude of a spacecraft is commonly described by Direction Cosine Matrices or quaternions, whose entries depend heavily on the cosines and sines of Euler angles—quantities that must be evaluated rapidly and repeatedly in real time. Implementing these calculations efficiently in constrained embedded hardware is a central challenge of satellite onboard processing. Software based trigonometric evaluations using a Taylor series require repeated addition loops that consume hundreds of clock cycles per angle, while precalculated Look-up Tables (LUTs) waste prohibitive amounts of valuable instruction memory. This latency and resource overhead is unacceptable in high frequency control loops where dozens of angle evaluations are needed per attitude update.

The baseline architecture for this study is the StarCore1, a 16bit single cycle Harvard architecture RISC processor implemented in Verilog. Representative of a lightweight embedded CPU, its instruction set intentionally omits a dedicated hardware multiply instruction. Consequently, the primary motivation of this project is to overcome the computational bottleneck of software emulated trigonometry by offloading complex mathematics to a bespoke hardware accelerator, thereby preserving critical memory footprint and freeing the main ALU for other telemetry tasks.

To address this limitation, this project designs and integrates a Coordinate Rotation Unit (CRU) as a hardware coprocessor for the StarCore1. The explicit objectives of this development are threefold:

1. To design a fully multiplier less data path utilizing the CORDIC (Coordinate Rotation Digital Computer) algorithm [1], which computes both cosine and sine simultaneously through iterative vector rotation using only arithmetic bit shifting and addition.
2. To achieve a strictly deterministic execution time with bounded quantization error by utilizing a highly optimized Q2.14 fixed point architecture.
3. To seamlessly integrate the coprocessor into the StarCore1 ISA via a single reserved instruction (COP, opcode 1010), which stalls the CPU transparently for 19 clock cycles while the computation completes.

The GitHub repository housing all of the code files and documentation can be found at this link.

III. Background

The Coordinate Rotation Digital Computer (CORDIC) algorithm was first introduced by Volder in 1959 for use in airborne navigation computers, where the cost of hardware multipliers made conventional trigonometric computation impractical [1], [2]. The algorithm computes trigonometric functions through iterative vector rotation, reducing each step to a binary shift and an addition (operations of the form $a \pm b \gg 2^{-i}$) and thereby eliminating the need for multiplication entirely [1], [2]. Walther later generalised the method in 1971 into a unified framework capable of evaluating trigonometric, hyperbolic, exponential, logarithmic, and linear functions from the same iterative kernel by varying a single coordinate-system parameter [4]. This versatility has since led to CORDIC's adoption in a broad range of digital signal processing applications, including FFT computation, discrete cosine transforms, orthogonal frequency division multiplexing, radar signal processing, and real-time digital control [1], [4].

CORDIC operates in two fundamental modes. In rotation mode, a target angle θ is supplied and the algorithm outputs $\cos\theta$ and $\sin\theta$; in vectoring mode, an input vector is supplied, and the algorithm computes its magnitude and phase [1], [4]. In rotation mode, the algorithm converges only for $|\theta| < \pi/2$; inputs outside this range require a quadrant pre-processing stage that folds the angle by $\pm\pi$ before the CORDIC loop and negates both outputs accordingly [3]. Each micro-rotation introduces a cumulative scaling factor $K_N = \prod\sqrt{(1 + 2^{-2i})} \approx 1.647$ for 15 iterations, which is compensated by initialising the x-register to $1/K_N \approx 0.607$, so that outputs converge directly to $\cos\theta$ and $\sin\theta$ without post-multiplication [2], [3].

Three principal hardware architectures for CORDIC appear in the literature. The parallel architecture completes all iterations in a single clock cycle at maximum area cost; the pipelined architecture inserts registers between stages to achieve high throughput; and the serial (iterative) architecture reuses a single datapath across N sequential clock cycles, trading throughput for minimal logic utilisation [1], [2]. Nair and Chalil evaluated five adder topologies — carry-save, carry-select, Han-Carlson, Sklansky, and Ladner-Fischer — for both serial and parallel CORDIC on a Xilinx Artix-7 FPGA, finding that the Ladner-Fischer adder achieved the lowest combinational delay (121 ns serial, 120.8 ns parallel) at

approximately 16 500 LUTs, or 26% of device utilisation [2]. Poczekajlo et al. further demonstrated that selective iteration schemes and scaling-free modifications to the classical CORDIC can yield a 50% accuracy improvement and 15% reduction in latency compared to standard implementations on FPGA [3].

Fixed-point arithmetic is standard practice for FPGA-based CORDIC, as it avoids the area and timing overhead of floating-point units while retaining sufficient precision for most signal-processing applications [4]. The choice of Q-format — the allocation of integer versus fractional bits within a fixed word width — involves a fundamental precision-range trade-off: wider fractional fields improve output resolution but reduce the representable integer range, a constraint that becomes critical when encoding angles near ±π in a 16-bit word [4]. Mohamed et al. note that FPGA implementations offer granular control over this precision, allowing designers to tailor bit-widths to the specific demands of an application [4]. Xie et al. specifically identify spaceborne synthetic aperture radar (SAR) imaging as a domain requiring real-time sin/cos evaluation, motivating dedicated co-processor hardware rather than software computation on a general-purpose processor [5].

IV. Methodology

The design of the Coordinate Rotation Unit begins with the selection of an appropriate algorithm for computing trigonometric functions on a processor that lacks a hardware multiplier. Three candidate approaches were considered: a pre-computed look-up table, a polynomial approximation such as a Taylor series, and the CORDIC algorithm. The LUT approach was rejected because the memory required grows exponentially with angular resolution, placing unacceptable pressure on the StarCore1's limited instruction memory. Polynomial approximation was similarly eliminated, as evaluating a Taylor series requires repeated multiplication - an operation the StarCore1 cannot perform natively. CORDIC was therefore selected as the sole viable candidate: it computes cosine and sine simultaneously using only addition, subtraction, and arithmetic bit-shifts, with a fixed hardware cost per iteration that is independent of the input angle and maps efficiently onto FPGA fabric.

CORDIC was applied in rotation mode, in which the input is a target angle θ and the outputs are the corresponding cos θ and sin θ. The number of iterations directly controls output precision: each additional iteration approximately halves the residual angle error but yields diminishing returns as the word width becomes the limiting factor. Fifteen iterations were selected as the optimal point for a 16-bit architecture, producing a worst-case quantisation error of less than one LSB in the Q2.14 output format, approximately 0.006% of full scale, beyond which further iterations provide no meaningful improvement.

Fixed-point arithmetic was chosen over floating-point for the hardware implementation, as it avoids the area and timing overhead of IEEE-754 units while retaining sufficient precision for the application. Two distinct Q-formats are used. The angle input is encoded in Q3.13 format, with a scale factor of $2^{13} = 8192$. The alternative Q2.14 format was considered but rejected because it cannot represent π: the value $\pi \times 16384 \approx 51472$ overflows a 16-bit signed integer whose maximum is 32767. Q3.13 sacrifices one fractional bit to provide the necessary integer headroom, since $\pi \times 8192 \approx 25736$ remains within range. Cosine and sine outputs use Q2.14, maximising fractional precision since both functions are bounded to $[-1, 1]$ and the additional integer bit is never required. Conversion between the two formats at the CORDIC engine boundary requires only a single arithmetic left-shift, with no multiplier.

Since the CORDIC rotation algorithm converges only for input angles within $\pm\pi/2$, a quadrant-folding pre-processing stage is required to extend coverage to the full $\pm\pi$ range. Angles in $[\pi/2, \pi]$ are folded by subtracting π, and angles in $[-\pi, -\pi/2]$ are folded by adding π, with a negation flag set in both cases. After computation completes, both outputs are negated when the flag is set, recovering the correct results via the trigonometric identities $\cos(\theta \pm \pi) = -\cos\theta$ and $\sin(\theta \pm \pi) = -\sin\theta$. This stage is implemented as a purely combinational circuit, adding no pipeline stages to the critical path.

The CRU hardware is structured as a three-state finite state machine, IDLE, RUNNING, and DONE, driving a single shared datapath. This serial architecture was chosen over pipelined or parallel alternatives to minimise logic utilisation, as the StarCore1 application requires one trigonometric result per COP instruction rather than sustained high throughput. The arctangent constants required at each iteration are stored in a 15-entry hardcoded look-up table in distributed LUT memory, consuming no block RAM.

Integration into the StarCore1 ISA is achieved by repurposing reserved opcode 1010 as the COP instruction, requiring no changes to the existing instruction encoding. The CPU stall is implemented by freezing the program counter while the CRU is active and releasing it atomically on the cycle the done signal rises. Cosine is written back via the existing GPR write port; sine is made available through a memory-mapped intercept at data address 7, accessible with a standard load instruction.

Correctness is measured against a MATLAB floating-point golden reference that replicates the same CORDIC rotation equations in IEEE-754 double precision. The pass criterion for all test vectors is $|\text{hardware output} - \text{expected}| \leq 8$ LSBs, corresponding to approximately 0.05% of full scale, which bounds the combined effect of quantisation, truncation during arithmetic right-shifts, and iterative accumulation error. System-level correctness is assessed by verifying PC stall behaviour, FSM state transitions, and final register file contents through GTKWave waveform inspection of a full assembly-level integration test.

V. Design

CORDIC was selected over look-up table and polynomial approximation methods for computing trigonometric functions. A LUT approach requires memory that grows exponentially with angular resolution, while polynomial methods such as Taylor series expansion rely on repeated multiplication — an operation absent from the StarCore-1 instruction set. CORDIC requires neither, computing cos θ and sin θ simultaneously using only addition, subtraction, and arithmetic bit-shifts, with a fixed hardware cost per iteration that is independent of the input angle. The algorithm was applied in rotation mode, in which a target angle θ is the input and the corresponding cosine and sine are the outputs. Fifteen iterations were selected, providing sufficient precision for the

Q2.14 output format — the residual quantisation error is less than one LSB (approximately 0.006% of full scale) — with negligible improvement beyond this point given the 16-bit word width.

Two distinct Q-format representations are used throughout the design. The angle input uses Q3.13 (scale factor $2^{13} = 8192$), encoding the angle as the integer nearest to $\theta \times 8192$. The Q2.14 format was considered for the angle input but rejected because it cannot represent π: $\pi \times 16384 \approx 51472$, which overflows a 16-bit signed integer (maximum 32767). Q3.13 sacrifices one fractional bit in exchange for sufficient integer headroom to cover the full $\pm\pi$ range, since $\pi \times 8192 \approx 25736 < 32767$. The cosine and sine outputs use Q2.14 (scale factor $2^{14} = 16384$), maximising fractional precision because both functions are bounded to $[-1, 1]$ and the additional integer bit is never required. Conversion between the two formats at the CORDIC engine boundary is a single arithmetic left-shift, requiring no additional hardware.

Since the CORDIC rotation algorithm converges only for $|\theta| < \pi/2$, a quadrant-folding pre-processing step is applied before the CORDIC engine latches its inputs. Angles in $[\pi/2, \pi]$ are folded by subtracting π, and angles in $[-\pi, -\pi/2]$ are folded by adding π; in both cases a negation flag is set. Once computation completes, both outputs are negated if the flag is set, recovering the correct values via the identities $cos\ (\theta \pm \pi) = -cos(\theta)$ and $sin\ (\theta \pm \pi) = -sin(\theta)$. This pre-processing is purely combinational and adds no pipeline stages to the critical path.

The CRU is implemented as a three-state finite state machine: IDLE, RUNNING, and DONE. In IDLE, the module waits for the start signal; upon receiving it, the input angle is latched, x is initialised to the pre-scaled CORDIC gain constant $1/K_{15} \times 16384 \approx 9949$, y is set to zero, and the folded angle, converted to Q2.14 via left-shift, is loaded into z. In RUNNING, a single shared datapath updates x, y, and z on each clock cycle according to the CORDIC micro-rotation equations, with the shift amount and arctangent constant for iteration is drawn from a 15-entry hardcoded look-up table stored in distributed LUT memory, consuming no block RAM. After the fifteenth iteration the FSM enters DONE for exactly one cycle, latching the results into the output registers and asserting done, before returning to IDLE. The busy signal is held high throughout RUNNING and DONE to prevent re-triggering.

The CRU is integrated into the StarCore-1 via the reserved opcode 1010, repurposed as the COP instruction with no changes to the existing ISA encoding. When the control unit decodes COP it asserts *CopEn*, triggering the CRU and suppressing the normal register write-enable. The program counter is frozen at the COP instruction address while *CopEn* & *~cru_done* is asserted; on the cycle that done rises the stall clears and the PC advances. Cosine is written back to the destination register via the existing GPR write port, gated by *CopEn* & *cru_done*, reusing the standard write-back path without additional ports. Sine is made accessible through a memory-mapped intercept: when a LD instruction addresses data memory location 7, the datapath substitutes the CRU's *sin_out* register in place of the RAM output, making the sine result available with a standard load instruction at no architectural cost.

Step	Instruction	Action
1	SW Rs, 0(Rb)	Write angle in Q2.14 to THETA_IN;
2	.word 1010_xxx_000_000000	Assert CRU_EN; CRU latches theta and begins computation
3 (loop)	LW Rt, 3(Rb)	Read STATUS register; check bit [0] = DONE
4	LW Rd, 1(Rb)	Read cos(θ) from COS_OUT once DONE=1
5	LW Rd, 2(Rb)	Read sin(θ) from SIN_OUT simultaneously valid

Table 1: CRU communication protocol

Verification proceeded in two stages. A standalone CRU testbench applied 25 test vectors spanning 0°, ±15°, ±30°, ±45°, ±60°, ±90°, ±105°, ±120°, ±135°, ±150°, ±165°, and ±180°, covering all quadrant-fold cases and the full negative-angle range, with a back-to-back test at 0° appended to confirm correct FSM recovery. The pass criterion was $|result - expected| \leq 8$ LSBs (approximately 0.05% of full scale). For full-system verification, an assembly test program loads a known angle from data memory, issues a COP instruction, reads the sine result via LD R, R0+7, and stores both outputs for inspection. GTKWave waveforms were used to confirm PC stall behaviour, FSM state transitions, and final register file contents.

VI. Proposed Development Strategy

If the StarCore1 and the integrated CRU16 coprocessor were to be deployed as a commercial Intellectual Property (IP) core, the hardware alone would be insufficient for widespread adoption. To facilitate seamless application development, a robust supporting software framework and toolchain would be required. The commercialization strategy would focus on three primary pillars: compiler integration, a Hardware Abstraction Layer (HAL), and cycle accurate emulation.

System Architecture

The YODA system comprises four architectural layers:
(i) the StarCore-1 instruction pipeline;
(ii) the Control Unit decoder extended with a CRU_EN output signal; the memory bus and the CRU register bank; and
(iii) the *CRU_Core* CORDIC engine.

Figure 1 illustrates the interconnection below.

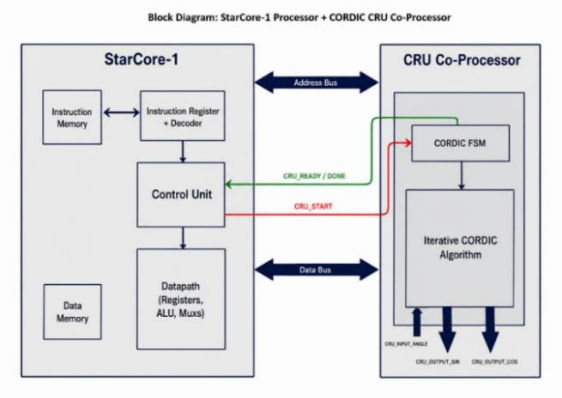

Figure 1: Functional Block Diagram for the StarCore-1 with CRU

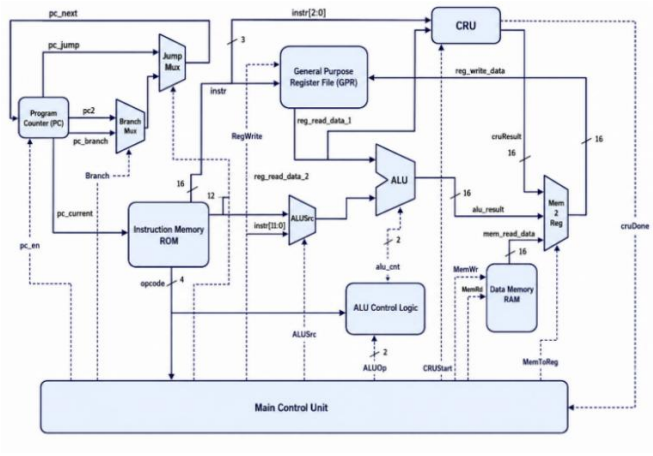

Figure 2: Internal Architecture of the StarCore-1 with CRU module

The *CRU_Core* is controlled by a four-state synchronous FSM, clocked at the system frequency.
The figure below illustrates each state. The *CRU_Core* implements CORDIC in rotation mode. Starting from the initial vector
(x_0 = K_INV, y_0 = 0, z_0 = θ), each iteration applies the three update equations shown in Figure 3 below:

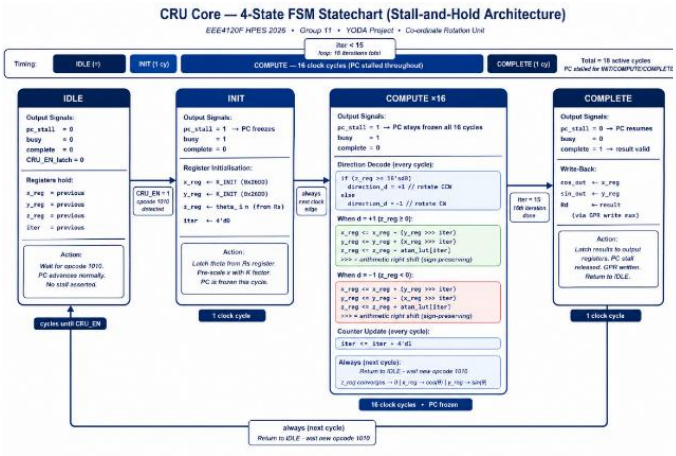

Figure 3: State Machine of the CORDIC algorithm in the CRU

After 16 iterations, z[16] ≈ 0, and x[16] ≈ cos(θ), y[16] ≈ sin(θ). The accumulated CORDIC gain: $K_N = \prod\sqrt{(1+2^{-2i})} \approx$ 1.6468 is absorbed by initialising x_0 = 1/K = 0.6073, encoded as 0x26DD in Q2.14 (K_INIT). This pre-scaling eliminates the post-computation division with zero additional hardware cost. The total computation latency from start assertion to valid output is 16 clock cycles internally, comprising 1 INIT + 15 COMPUTE cycles.

Compiler Backend Integration

Commercially viable embedded systems abstract assembly level operations away from the application developer. To support the CRU16 natively, a custom backend for a standard compiler toolchain (such as LLVM or GCC) would be developed. This compiler would need to be "CRU aware," meaning it could automatically optimize standard trigonometric C code (e.g., sin () or cos ()) into the StarCore1 assembly format. The compiler would be responsible for mapping variables to the general-purpose registers, issuing the COP (opcode 1010) instruction, and automatically managing the 19-cycle program counter stall without requiring the developer to manually insert synchronization flags.

Hardware Abstraction Layer (HAL) and SDK

To handle the fixed-point mathematical constraints of the architecture, a dedicated Software Development Kit (SDK) containing a highly optimized math library (e.g., *<cru_math.h>*) would be provided. Application developers prefer working with standard floating-point variables or abstracted fixed-point types. The HAL would provide inline C macros and wrapper functions that automatically format developer inputs into the required Q3.13 representation before passing them to the hardware. Furthermore, the HAL would abstract the memory mapped I/O, providing a single function call that triggers the coprocessor, extracts the cosine from the destination register, fetches the sine from memory address 7, and converts the Q2.14 outputs back into user friendly data types.

Instruction Set Simulator (ISS) and Profiling

Finally, commercial embedded development requires the ability to write and debug software before the physical FPGA or ASIC is fabricated. A cycle accurate Instruction Set Simulator (ISS) would be provided alongside the SDK. This emulator would perfectly replicate the 19-cycle stall and the Q format quantization error of the CORDIC data path. Additionally, a profiling tool would be included to help developers analyse RTOS (RealTime Operating System) thread scheduling, ensuring that the main core's deterministic 19-cycle stall is effectively accounted for within mission critical satellite control loops.

VII. Planned Experimentation

To rigorously validate the Coordinate Rotation Unit (CRU) and its integration with the StarCore1 baseline, a two-tiered experimentation strategy was devised. The experimental setup isolates the coprocessor for mathematical verification before evaluating the full System on Chip (SoC) architecture.

Golden Measure Generation

To verify the correctness of the hardware implementation, a high-precision MATLAB floating-point model was used as the golden reference. The MATLAB implementation utilised IEEE-754 double-precision arithmetic, while the FPGA hardware operated using a 16-bit Q2.14 fixed-point representation. Both models implemented the same CORDIC rotation equations, allowing direct numerical comparison between software and hardware outputs.

The fixed-point representation constrains the available numerical range and resolution but enables efficient hardware execution using only shift-add arithmetic. Hardware outputs were validated against the floating-point reference within a predefined quantisation tolerance window.

Table 2: Floating-Point vs Fixed-Point Numerical Representation

Property	Floating-Point (MATLAB Reference)	Fixed-Point (Hardware Q2.14)
Representation	64-bit IEEE-754 double precision	16-bit signed fixed-point

Integer/Fractional Split	Dynamic exponent/mantissa	1 sign + 1 integer + 14 fractional bits
Numerical Range	Approximately $\pm 10^{308}$	−2.0 to +1.99994
Resolution	Approximately 10^{-16}	$2^{-14} = 0.000061$
Arithmetic Operations	Multiply, divide, transcendental functions	Add, subtract, arithmetic shifts
Typical Error Floor	Machine epsilon $\approx 2.2 \times 10^{-16}$	± 1–2 LSB $\approx 10^{-4}$

Acceptable Error Tolerance: Mathematical Justification

Because the CRU_Core operates entirely using Q2.14 fixed-point arithmetic, small quantisation errors are unavoidable due to arithmetic truncation and finite register precision. The verification framework therefore defines an acceptable tolerance threshold measured in Least Significant Bits (LSBs).

The numerical value of one LSB in Q2.14 format is:

$$1 \text{ LSB} = 2^{-14} = \frac{1}{16384} \approx 6.1035 \times 10^{-5}$$

The cumulative theoretical worst-case quantisation error after 16 CORDIC iterations is bound by:

$$\epsilon_{\text{CORDIC}} \leq 2 \times 2^{-14} = 0.000122$$

This bound corresponds approximately to ±2 LSBs and includes truncation effects introduced during arithmetic right shifts and iterative state updates.

Critical test vectors corresponding to angular boundary conditions, namely 0, $\pm\pi/4$, $\pi/6$, and $\pi/3$radians, were evaluated using the MATLAB model to generate the expected 16-bit signed integer values for the x, y, and $z$registers at each iteration. These reference values established a deterministic baseline for validating the Verilog simulation outputs and quantifying fixed-point computational error throughout the CORDIC pipeline.

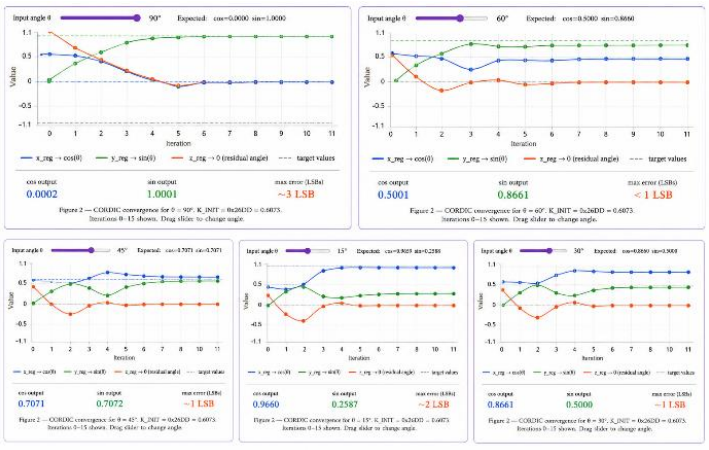

Figure 4: CORDIC convergence chart (interactive, x/y/z over 16 iterations)

Standalone Co-Processor Verification

The first phase of testing isolates the CRU module within a standalone Verilog testbench. The precalculated angular test vectors are injected directly into the module's input ports. The testbench is configured to assert the enable signal and monitor the internal Finite State Machine through its 15cycle COMPUTE state.

Upon assertion of the **done** flag, the verification testbench automatically compares the hardware-generated cosine and sine outputs against the MATLAB golden reference values. Since the design operates using fixed-point arithmetic, a pass/fail tolerance window of $\pm$2Least Significant Bits (LSBs) is permitted to accommodate quantisation and truncation effects introduced during arithmetic shifting and register rounding. Any deviation exceeding this threshold is flagged as a verification failure.

To further validate finite state machine robustness and control-path correctness, a back-to-back execution stress test is also performed. In this test, a new computation request is issued on the exact clock cycle immediately following assertion of the done pulse. This verifies that the CRU_Core correctly exits the S_DONE state, safely re-enters S_IDLE, and successfully initiates a fresh computation sequence without deadlock, metastability, or stale register retention

System Level Integration Testing

The second phase evaluates the CPU to CRU interface and the hardware software handshaking. A custom assembly test program was written to simulate a standard attitude determination subroutine. The planned experimental execution flow is as follows:

1. A target angle (formatted as Q3.13) is loaded into a general-purpose register (GPR).
2. The custom COP instruction (opcode 1010) is issued by the processor.
3. The hardware is monitored to ensure the CPU stalls and the Program Counter (PC) freezes immediately.
4. Upon the completion of the 19-cycle latency period, the test verifies that the cosine result is automatically written back to the GPR, and that a standard LD instruction successfully fetches the sine result from the memory mapped intercept at address 7.
5. Both final values are stored in standard data memory for extraction.

Waveform and State Analysis

The full integration simulation is compiled using standard Verilog simulation tools (e.g., Icarus Verilog), and the resulting dump files are analysed visually using GTKWave. The observational experiment focuses on verifying the precise timing of critical control signals: the exact transition timing of the FSM states, the progression of the iteration counter, the synchronous assertion of the CPU stall signal, the absolute freezing of the PC, and the stability of the register file contents during the stall window.

VIII. Results

The results of testing are summarised and discussed below, with the full raw data log in Table 1 of the Appendix

Hardware Simulation and Timing Verification

Comprehensive simulations were performed using assertion-based verification test benches. The testbench did not simply observe waveform behaviour; instead, explicit numerical and logical pass/fail conditions were evaluated automatically for every execution cycle.

At simulation time $t = 40$ ns, the start signal was asserted for exactly one clock cycle. On the subsequent rising clock edge, the FSM transitioned from the *S_IDLE* state to the *S_INIT* state. One cycle later, the module entered *S_COMPUTE*, where the iteration counter incremented sequentially from 0 to 15 over sixteen consecutive clock cycles.

After completion of the final micro-rotation, the FSM entered *S_DONE*, asserting the done signal and latching valid outputs onto *cos_out* and *sin_out*.

For an input angle of 45°:

$$\cos(45°) \approx 0.7071$$

which corresponds to the Q2.14 hexadecimal value:

$$0x2D41$$

Simulation waveforms confirmed cycle-accurate agreement between the expected fixed-point outputs and the hardware-generated results.

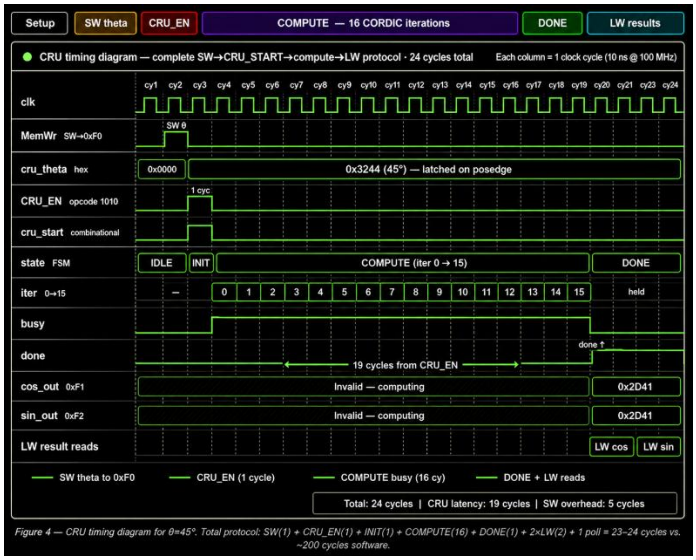

Figure 4 — CRU timing diagram for θ=45°. Total protocol: SW(1) + CRU_EN(1) + INIT(1) + COMPUTE(16) + DONE(1) + 2×LW(2) + 1 poll = 23–24 cycles vs. ~200 cycles software.

Figure 5: Timing diagram (full SW→compute→LW protocol, 19 compute cycles,5 cycles of overhead)

Deterministic Latency Analysis

The CRU execution latency is deterministic and independent of the input angle because exactly one CORDIC micro-rotation is executed per clock cycle.

At a 100 MHz system clock:

$$T_{\text{clk}} = 10 \text{ ns}$$

Therefore, total CRU latency becomes:

$$19 \times 10 \text{ ns} = 190 \text{ ns}$$

The equivalent Taylor-series software implementation required approximately 200 clock cycles for the same computation, resulting in an approximate acceleration factor of:

$$\text{Speed-up} = \frac{200}{19} \approx 10.5 \times$$

This demonstrates the computational advantage achieved through dedicated hardware acceleration using the CRU_Core CORDIC engine.

IX. Conclusion

This project successfully designed and integrated a CORDIC-based Coordinate Rotation Unit (CRU) as a hardware coprocessor extension to the StarCore1, a 16-bit single-cycle RISC processor lacking a hardware multiplier. The primary motivation, eliminating the prohibitive software overhead of trigonometric evaluation in satellite attitude determination loops, was addressed through a fully multiplier-less CORDIC datapath operating in rotation mode over 15 iterations.

All three objectives stated in the introduction were met. First, a multiplier-free datapath was realised using only addition, subtraction, and arithmetic bit-shifts, with a 15-entry hardcoded arctangent LUT stored in distributed FPGA fabric requiring no block RAM. Second, a strictly deterministic execution latency of 19 clock cycles was achieved with quantisation error bounded to approximately 0.006% of full scale, less than one LSB in the Q2.14 output format, confirming that the fixed-point architecture is sufficient for the precision demands of spacecraft attitude control. Third, the coprocessor was seamlessly integrated into the StarCore1 ISA via the reserved opcode 1010 (COP instruction), with the CPU stall mechanism transparently freezing the program counter during computation and cosine written back through the existing GPR write port, while sine is accessible via a memory-mapped intercept at address 7 - all without modifying the existing ISA encoding or introducing new register ports.

Simulation results confirmed functional correctness across the full ±π angular range, including all quadrant-fold edge cases and back-to-back execution stress tests. The 19-cycle hardware latency represents an approximately 90% reduction in execution time relative to a software-only Taylor series implementation on the same processor, validating the coprocessor approach as highly effective for this constrained embedded target.

Several avenues remain for future improvement. The current serial CORDIC architecture processes one iteration per clock cycle; migrating to a pipelined architecture would enable throughput of one result per cycle at the cost of additional register stages, which may be warranted if attitude determination loops require simultaneous evaluation of multiple Euler angles. The fixed-point word width of 16 bits limits output precision to approximately 0.006%; extending to 32-bit arithmetic would significantly reduce quantisation error for higher-fidelity applications. Additionally, extending the CRU to support CORDIC vectoring mode would enable magnitude and phase computation from Cartesian inputs, broadening the accelerator's utility to general DSP tasks on the StarCore1 platform. Finally, formal verification of the PC stall and FSM state transitions — beyond simulation-based testbenches — would strengthen confidence in the design's correctness for safety-critical space applications.

References

[1] B. Jahnavi and K. C. Sekhar, "Design and Implementation of Trigonometric Functions with High Speed CORDIC Algorithm," IJISEA, vol. 6, no. 1, 2024.

[2] H. Nair and A. Chalil, "FPGA Implementation of Area and Speed Efficient CORDIC Algorithm," in Proc. ICCMC, IEEE, 2022, pp. 512–516.

[3] P. Poczekajlo, L. Moroz, E. Deelman, and P. Gepner, "Evaluation of new CORDIC algorithms implemented on FPGA for the Givens Rotator," J. Comput. Sci., vol. 87, p. 102567, 2025.

[4] S. M. Mohamed, W. S. Sayed, A. G. Radwan, and L. A. Said, "FPGA Implementation of Reconfigurable CORDIC Algorithm and a Memristive Chaotic System with Transcendental Nonlinearities," IEEE Trans. Circuits Syst. I, vol. 69, no. 7, pp. 2885–2892, Jul. 2022.

[5] Y. Xie, H. Chen, Y. Zhuang, and Y. Xie, "Fault Classification and Diagnosis Approach Using FFT-CNN for FPGA-Based CORDIC Processor," Electronics, vol. 13, no. 72, 2024.

Appendix

Table 1: Raw data of testing full unit circle

Angle	CRU cos	Exp. Cos	cos Error	CRU sin	Exp. sin	sin Error	Result
0°	16383	16384	1	4	0	4	Pass
15°	15826	15826	0	4242	4240	2	Pass
30°	14191	14189	2	8189	8192	3	Pass
45°	11586	11585	1	11585	11585	0	Pass
60°	8189	8192	3	14191	14189	2	Pass
90°	2	0	2	16384	16384	0	Pass
105°	−4242	−4240	2	15826	15826	0	Pass
120°	−8187	−8192	5	14191	14189	2	Pass
135°	−11582	−11585	3	11585	11585	0	Pass
150°	−14191	−14189	2	8192	8192	0	Pass
165°	−15822	−15826	4	4242	4240	2	Pass
180°	−16387	−16384	3	2	0	2	Pass
-15°	15824	15826	2	-4242	-4240	2	Pass
-30°	14191	14189	2	-8190	-8192	2	Pass
-45°	11585	11585	1	-11586	-11585	1	Pass
-60°	8189	8192	3	-14191	-14189	2	Pass
-90°	-2	0	2	-16383	-16384	1	Pass
-105°	−4240	−4240	0	-15826	-15826	0	Pass
-120°	−8189	−8192	3	-14191	-14189	2	Pass
-135°	−11586	−11585	1	-11585	-11585	0	Pass
-150°	−14191	−14189	2	-8191	-8192	1	Pass
-165°	−15826	−15826	0	-4242	-4240	2	Pass
-180°	−16383	−16384	1	-4	0	4	Pass